Project Lab Level 2: The Night Watchman

Document: Project Lab Level 2: The Night Watchman
Version: v1.0.0
Author: Celaya Solutions
Contact: hello@celayasolutions.com
Date: 2026-09-06
SHA256: c46c501b66d49696b45344356ed24339b868a61e26880ee949d7b80f13af04f1
Chain: n/a
Tx: [not anchored]

Goal

Specify, run, and test a scheduled page watcher that records each round, alerts only on a meaningful change, and stops through a tested switch.

By the end of class, you can explain the trigger, predict each result, trace the saved value to an issue, and turn the watcher off. You will leave with a working project in your own fork and one private evidence file.

Start here

Complete Level 2 preparation before class. It covers the exact tools, downloads, folder, workflow installation, permissions, and account checks. Keep that page open beside this lesson. This meeting lasts 90 minutes; installations happen beforehand.

Use your own computer and existing course fork. Open `.zta/PROJECT-LAB-02.md` in a text editor and fill it as you work. Partner, paper, and saved examples are fallback routes; they do not prove an individual hosted launch. Stop after two failed attempts or five minutes on a single block and record the fallback.

Anchors / Anclas

- Look, compare, tell. / Mira, compara y avisa.

- If it cannot be turned off, it is not finished. / Si no se puede apagar, no está terminado.

- Autonomy is earned, not granted. / La autonomía se gana, no se concede.

Why this project matters

In Level 1 you asked a question and waited for an answer. Here a clock can start a small job while your laptop is closed. That change makes saved state, clear permission, useful alerts, and a stop control part of the project.

This watcher does not use a language model. You write the rule; ordinary code applies it. A coding assistant can help you understand the code, but it does not choose the alert rule during a run. A simple rule is enough when the signal is exact.

Imagine a workbench that can be OPEN or PAUSED. You want to hear when its status changes, not when someone edits a footer. The watcher reads only the text inside `id="watch-value"`. Blank, missing, duplicate, or unknown statuses are failures, not new workbench states.

Part	What it does here	Why it matters
Trigger	Manual **Run workflow**, or daily at 13:17 UTC while enabled	Names when a check may start
Look	Read the supplied page in your public fork at that run's commit	Prevents a changing cached address from obscuring the test
Compare	Compare OPEN or PAUSED with `.watch-state/watchman.json` on the default branch	A fresh runner can remember the last good result

Part	What it does here	Why it matters
Tell	Create one issue in the same fork for a valid change	Gives a human the old value, new value, and source
Receipt	Dated result in the run summary, step log, and 30-day downloadable artifact	Lets you check what actually happened
Switch	Disable the workflow, set the enable variable false, and cancel any queued or active run	Stops future checks and deals with work already underway

The first look saves a baseline without an issue. Later identical looks say `no change`. The first OPEN-to-PAUSED transition creates one issue. A repeat PAUSED run creates none. A failed request keeps the last good value and returns a red run with `decision: failed`.

The workflow explicitly uses UTC. Daily 13:17 UTC is 06:17 at UTC-7 or 07:17 at UTC-6. Schedules can be delayed or dropped; do not promise an exact alert minute. Today's tests use the manual control. GitHub schedule reference

Safety stop / Alto de seguridad

Use only the supplied public page in your learner fork for the core route. No private pages, login bypass, personal tracking, arbitrary addresses, or aggressive polling. `GITHUB_TOKEN` is supplied for the job by GitHub; never create or paste a personal token.

The code writes one state file and may create the explicitly allowed practice issue in that same fork. The token's contents/issues scopes cover the repository, not just that file; code review and a disposable learner fork keep that authority understandable. Do not enable write access for fork pull requests or use an employer repository. GitHub permission reference

Never buy, send replies, delete content, post outside this practice issue, or spend money automatically. Public issues and logs contain only the supplied status and project receipts; put your personal learning notes in the private worksheet.

Vigila solamente la página de práctica. La única publicación permitida es el aviso de práctica en tu propio repositorio. No compres, respondas, borres ni publiques en otro lugar.

Task 1: Write the watchman spec

Step 1. Name the exact job before enabling it

Where: Your worksheet and your fork's Code tab. **Action:** Open `courses/project-lab/level-02/assets/practice-page.html`, click **Raw**, and copy that address into the spec. Fill all six lines:

1. Address: the raw practice page in my fork; each run reads its fixed commit version.

2. Signal: the text inside the one element marked `watch-value`.

3. Meaningful change: OPEN changes to PAUSED, or PAUSED changes to OPEN. Other text is a failure.

4. Trigger: manual runs in class; daily at 13:17 UTC only while enabled; leave disabled after class.

5. Alert: one issue in my own practice fork naming the old value, new value, and source.

6. Never list: no purchase, reply, deletion, posting elsewhere, private access, personal tracking, or automatic spending.

Why: A written rule makes the result testable. **Expected result:** A partner predicts baseline = no issue, same status = no issue, valid changed status = one issue. **Recovery:** If you disagree, point to the exact line and fix the spec before touching the switch.

Step 2. Inspect the project and predict the first run

Where: Code tab, then worksheet. **Action:** Open `projects/watchman/watch.py` and locate `extract_value`, `run_round`, and `hosted_context`; use the assistant/manual card for a plain-language explanation. Confirm the practice page says OPEN. Write the baseline prediction and current time before running.

Why: You should know what you are authorizing. **Expected result:** You can name the allowed issue, state file, and switch. **Recovery:** A wrong page or existing state needs instructor review. Do not delete state to make an unexplained result pass.

Task 2: Run, change, and inspect

Step 3. Enable this fork deliberately

Where: Your fork's **Settings > Secrets and variables > Actions > Variables**. **Action:** Click **New repository variable**; name it `WATCHMAN_ENABLED`, value `true`, then save. If it already exists, edit it. Open **Actions > Project Lab Watchman**; click **Enable workflow** if it was disabled.

Why: This authorizes the practice job. **Expected result:** The workflow is available and the job's enable check can pass. **Recovery:** A gray/skipped job is not a baseline. Check exact spelling, lowercase `true`, the workflow's enabled state, and fork ownership. This variable is not a secret or API key.

Step 4. Run the baseline and read its receipt

Where: Actions > Project Lab Watchman > Run workflow. **Action:** Select your fork's default branch, then the green **Run workflow** button. Refresh the list until a new run appears. Open it, then the **watch** job and **Look, compare, tell** step. Wait for completion; do not start another run while one is pending or active.

Why: One completed run gives one result to judge. **Expected result:** Green run, `decision: baseline saved`, `current: OPEN`, `previous: null`, and no new issue. The Code tab now contains `.watch-state/watchman.json` with `current: OPEN` and `pending: null`. **Recovery:** A red run is a failure. Copy only the error category and timestamp to your worksheet, then use the failure table below.

Where: Run summary and worksheet. **Action:** Copy the run URL, time, source commit, previous/current values, decision, and verdict. Under **Artifacts**, download `watchman-…`; unzip it and open `rounds.jsonl` with a text editor. Each JSON line is one receipt, not a program to run.

Why: A green badge alone does not explain what happened. **Expected result:** The artifact and run summary agree. **Recovery:** If checkout or setup failed before the watcher started, there may be no artifact; the GitHub job log is the failure receipt. Save useful receipts before the 30-day artifact expiry. GitHub artifact guide

Step 5. Run unchanged

Where: Worksheet, then the same Actions control. **Action:** Write `Expected: no change; no new issue` with the time. Leave the page unchanged. Use **Run workflow** to create a new run on the default branch; wait and record its receipt as before.

Why: Silence is part of the alert rule. **Expected result:** `decision: no change`, previous OPEN, current OPEN, and no new issue. **Recovery:** A failure is not no change. A second baseline indicates missing/wrong state; stop and check the fork, branch, and file before proceeding.

Step 6. Make one controlled change

Where: Worksheet, then your fork's **Code** tab on its default branch. **Action:** Write `Expected: OPEN to PAUSED; exactly one new issue`. Open the practice-page path again, click the pencil (**Edit this file**), and change only:

```
<strong id="watch-value">OPEN</strong>
```

to:

```
<strong id="watch-value">PAUSED</strong>
```

Click **Commit changes**, enter `Pause the practice workbench`, select a direct commit to your fork's default branch when permitted, and confirm. If branch rules require a pull request, ask the instructor to review that route; the page change must reach the default branch before this test.

Why: One deliberate input change makes the outcome explainable. **Expected result:** The Code tab shows PAUSED at a new commit. **Recovery:** If you changed the marker, added another marker, or edited the wrong branch, correct that one edit before starting the run.

Step 7. Inspect the changed result and check a repeat

Where: Actions. **Action:** Use a fresh **Run workflow** on the default branch. Do not choose **Re-run jobs** from an older run; it uses the older source commit. After completion, open the receipt's issue link or the **Issues** tab.

Why: The new run must read the page commit you just made. **Expected result:** `decision: changed`, old OPEN, new PAUSED, and one issue titled `Change spotted: OPEN to PAUSED`. The issue body includes both values and the source. State now says PAUSED with no pending alert. Email delivery is not assessed.

Recovery: If the issue exists but the run failed while saving, preserve it and start one new run after the state-write block is fixed. `alert recovered` means the existing issue was found and state was completed. It is not a second alert. If delivery is uncertain, disable and follow the instructor's recovery card; do not delete the pending record.

Where: Worksheet and Actions. **Action:** Predict `PAUSED unchanged; no additional issue`, then start one more new run. Record its verdict and compare the issue count. **Why:** Repeats should not create noise. **Expected result:** No change, still one issue for this transition. **Recovery:** Stop if a second issue appears and preserve both links for review.

Task 3: Use the switch

Step 8. Stop future triggers and finish active work

Where: Actions > Project Lab Watchman. **Action:** Wait for your test run to finish. Open the workflow's three-dot menu and choose **Disable workflow**. In repository Variables, change `WATCHMAN_ENABLED` to `false`. Return to Actions; cancel any still queued or active watcher run using **Cancel workflow** on that run's page and wait for its final state.

Why: Disabling prevents new triggers; it does not undo work already underway. **Expected result:** The workflow shows disabled, the variable is false, and no watcher run is queued or active. **Recovery:** If you lack write access, the fork owner must use the controls. Keep the block visible in your evidence. GitHub disable guide, cancel guide

Step 9. Test the stopped state

Where: Refreshed workflow page and worksheet. **Action:** Try to reach **Run workflow** again without clicking **Enable workflow**. Record the disabled banner, which control is absent/unavailable, the time, and that no new watcher run was created. Check the run list for outstanding jobs.

Why: Naming a switch is not using it. **Expected result:** Disabled workflow; manual trigger unavailable; no active or queued watcher work. **Recovery:** If you can still trigger it, confirm the correct workflow/fork and repeat the stop steps. Do not claim a future daily run was observed missing just because one minute passed. This check proves the current disabled state; an actual scheduled-run pilot is separate.

Keep it disabled after class. Turning off your laptop does not stop GitHub's hosted runner.

Quick check before proof

1. Where does the job run when your laptop is closed?

2. Why is `no change` different from `failed`?

3. Why does each run need a saved baseline?

4. What can the token do, and what does this code actually do?

5. Why use a fresh run after editing the page?

6. What does the disabled-state test prove, and what have you not observed?

Pass this level

A complete watchman spec; baseline, unchanged, and changed runs; one useful alert and log trail; a never list; and proof that the learner used the off switch.

Save `.zta/PROJECT-LAB-02.md` after completing every row, the repeat check, and the exit ticket. Keep honest Miss or Blocked verdicts; do not turn a saved example into live proof. In Desktop, fetch/pull the page and state commits made on GitHub so your clone catches up. Review any learner code edits and publish only those to your fork. Your private worksheet must not appear in Changes.

Where: Course sign-in. **Action:** Open Zero to Agent, then the Level 2 assessment. Choose your `PROJECT-LAB-02.md` file, complete the displayed acknowledgment if any, and submit. Open the resulting submission entry and view/download the file; check your nickname, final run URL, and stopped-state record. Follow the shared submission guide, selecting Level 2 and this filename.

Why: A local file is not a submission receipt. **Expected result:** A private receipt with the correct file and time. **Recovery:** If the assignment or upload control is missing, keep the file locally and tell the instructor. Record submission as Blocked until a real receipt exists; never put the evidence in a public issue to work around it.

If something fails

Symptom	What it means	Next action
Workflow missing	File/path/default branch or Actions enablement may be wrong	Return to preparation steps 3-4; inspect your fork
Gray/skipped job	Enable guard did not pass	Check variable spelling/value and selected repository
HTTP 401/403/404/422	Access, Issues, state write, or branch policy blocked a request	Read which step failed; ask the instructor; keep existing state and issue
Missing/empty/duplicate/unknown signal	Page failed validation	Restore exactly one OPEN or PAUSED marker; start a new run
Fetch timeout	No valid new observation	Keep old state; retry once after service recovery
State save failed before alert	No issue was sent	Repair state-write access, then start a new run
Issue exists; last save failed	Pending transition needs completion	Start a new run after repair; expect alert recovered, no duplicate
Alert delivery uncertain	The server may have created an issue	Disable; instructor checks pending ID before any resend
No email	Notifications differ from the issue itself	Grade the repository issue and log
Delay or outage	Hosted execution is unavailable	Use the labeled saved packet after two attempts/five minutes

Builder extension

After the core proof, use the manual/assistant card to change only the footer locally and show that the signal stays unchanged. Keep the same expected result and compare before/after receipts. This is an optional extension, not a required model call.

Write a Railway planning contract with start command, environment, one-round health result, saved state, durable storage, one schedule owner, stop control, and a dated cost estimate from current official pricing. Do not deploy, enter billing details, or add a second scheduler in this level.

Resumen en español

Prepara tu propia copia del curso. Escribe seis reglas y predice cada resultado antes de ejecutar. Guarda una línea base OPEN, ejecuta sin cambios, cambia a PAUSED y revisa un solo aviso. Repite sin cambios. Desactiva el flujo, cancela lo pendiente y comprueba el estado. Entrega PROJECT-LAB-02.md de forma privada. Una práctica guardada no demuestra una ejecución tuya.

Words for Level 2

- Baseline: the first valid status saved for comparison.
- Workflow: the file that tells GitHub which job to run.
- State: the last confirmed value and any unfinished alert.
- Receipt: a dated record of the observation and decision.
- Pending: an alert step that has not been fully completed.
- Off switch: the tested controls that stop new work and cancel outstanding work.